카담(CADAM) 프로젝트 — Cursor AI 바이브코딩 완전 초보 가이드

> **대상:** 코딩 경험 없는 비개발자
> **목표:** Cursor AI 설치부터 카담 프로젝트 첫 화면이 뜰 때까지
> **작성일:** 2026-03-11

🔵 Phase 0: 사전 준비물 설치 (Cursor 열기 전에 해야 할 것들)

Cursor AI는 "코드 편집기"입니다. 하지만 코드 편집기만으로는 웹사이트가 만들어지지 않습니다.
요리에 비유하면 Cursor는 "주방(도구)"이고, 아래 프로그램들은 "가스레인지, 냉장고, 수도(인프라)"입니다.

0-1. Node.js 설치

이것은 무엇인가?
JavaScript를 컴퓨터에서 실행할 수 있게 해주는 프로그램입니다.
Next.js(우리 프로젝트의 핵심 프레임워크)가 Node.js 위에서 돌아갑니다.

설치 방법:
1. https://nodejs.org 접속
2. **LTS** 버전 다운로드 (숫자가 짝수인 것, 예: 22.x.x)
3. 다운받은 파일 실행 → "Next" 계속 클릭 → 설치 완료

설치 확인 (터미널에서):
- Mac: `Cmd + Space` → "터미널" 검색 → 열기
- Windows: `Win + R` → `cmd` 입력 → 열기

```
node --version    # v22.x.x 같은 숫자가 나오면 성공
npm --version     # 10.x.x 같은 숫자가 나오면 성공
```

> **[왜 필요한가 — 사실]** PRD 기술 스택에 "Next.js 15"가 명시되어 있고,
> Next.js는 Node.js 런타임 위에서만 작동합니다.

0-2. Git 설치

이것은 무엇인가?
코드의 "버전 관리" 도구입니다. 문서 작업할 때 "최종.docx", "진짜최종.docx" 대신
변경 이력을 체계적으로 관리해줍니다.

설치 방법:
- Mac: 터미널에 `git --version` 입력하면 자동 설치 안내가 뜸
- Windows: https://git-scm.com 에서 다운로드 → 설치 (모든 옵션 기본값 OK)

설치 확인:

```
git --version     # git version 2.x.x 나오면 성공
```

0-3. Cursor AI 설치

이것은 무엇인가?
VS Code(마이크로소프트의 코드 편집기)를 기반으로 만든 AI 코딩 전용 에디터입니다.
채팅으로 AI에게 "이런 코드 만들어줘"라고 지시하면 코드를 생성/수정해줍니다.

설치 방법:
1. https://www.cursor.com 접속
2. "Download" 버튼 클릭 (Mac/Windows 자동 감지)
3. 다운받은 파일 실행 → 설치
4. 처음 실행 시 계정 생성 (구글 로그인 가능)

요금:
- Free: 월 2,000회 코드 자동완성 + 50회 느린 AI 요청
- Pro: 월 $20 — 무제한 자동완성 + 500회 빠른 AI 요청 (추천)

> **[추론]** 카담 프로젝트는 파일 50개 이상, 수백 번의 AI 요청이 필요하므로
> Pro 플랜을 사용하는 것이 현실적입니다.

🔵 Phase 1: 프로젝트 생성 (Next.js 초기화)

1-1. 프로젝트 폴더를 만들 위치 정하기

사고 과정:
- 바탕화면이나 Downloads 폴더는 피하는 것이 좋습니다 (경로에 한글/공백이 있으면 오류 가능)
- 추천 위치: `~/projects/` (Mac) 또는 `C:\projects\` (Windows)

터미널에서 실행:

```
# Mac
mkdir -p ~/projects
cd ~/projects

# Windows (CMD)
mkdir C:\projects
cd C:\projects
```

1-2. Next.js 프로젝트 생성

```
npx create-next-app@latest cadam
```

이 명령을 실행하면 질문이 나옵니다. 아래와 같이 답하세요:

```
✔ Would you like to use TypeScript?                 → Yes
✔ Would you like to use ESLint?                     → Yes
✔ Would you like to use Tailwind CSS?               → Yes
✔ Would you like your code inside a `src/` directory? → Yes
✔ Would you like to use App Router? (recommended)   → Yes
✔ Would you like to use Turbopack for next dev?     → Yes
✔ Would you like to customize the import alias?     → No (기본값 @/ 사용)
```

> **[사실]** PRD 기술 스택: "Next.js 15 (App Router) + TypeScript + Tailwind CSS"
> cursor_rules: "src/app/ 하위 Next.js App Router 규칙", "Tailwind CSS (인라인 style 금지)"
> 따라서 위 선택지는 PRD 요구사항을 그대로 반영한 것입니다.

완료 후 확인:

```
cd cadam
npm run dev
```

브라우저에서 `http://localhost:3000` 접속 → Next.js 기본 화면이 보이면 성공!
`Ctrl + C` 로 서버 중지.

🔵 Phase 2: Cursor AI에서 프로젝트 열기 & 환경 설정

2-1. Cursor로 프로젝트 열기

1. Cursor 실행
2. 상단 메뉴 → `File` → `Open Folder`
3. 방금 만든 `cadam` 폴더 선택
4. 왼쪽 사이드바에 프로젝트 파일 트리가 보이면 성공

2-2. Cursor AI 설정 최적화

Cursor Settings 열기:
- Mac: `Cmd + Shift + P` → "Cursor Settings" 검색 → 클릭
- Windows: `Ctrl + Shift + P` → "Cursor Settings" 검색 → 클릭

권장 설정:
- **Models:** `claude-sonnet-4-20250514` 또는 최신 Claude 모델 선택
- **Features → Composer:** 활성화 (멀티파일 동시 편집에 필수)
- **Features → Agent:** 활성화 (자동으로 터미널 명령 실행 가능)

> **[추론]** PRD가 매우 상세하므로 Claude 계열 모델이 긴 문서 이해에 유리합니다.
> GPT-4o도 사용 가능하지만, 한국어 PRD 처리는 Claude가 더 정확한 경향이 있습니다.

2-3. Cursor Rules 파일 배치 (가장 중요!)

이것은 무엇인가?
Cursor AI에게 "이 프로젝트에서는 이런 규칙을 지켜라"고 알려주는 설정 파일입니다.
사람으로 치면 "프로젝트 온보딩 문서"와 같습니다.

Cursor의 터미널 열기: 하단 패널에서 `Terminal` 탭 클릭 (없으면 `Ctrl + ` `)

방법 1 — 터미널 명령어로 생성:

```
# 프로젝트 루트에서 실행

# 1) .cursorrules 파일 생성 (레거시 호환)
# → dot_cursorrules.txt 내용을 복사 붙여넣기

# 2) .cursor/rules 파일 생성 (최신 방식)
mkdir -p .cursor
# → cursor_rules.md 내용을 복사 붙여넣기

# 3) docs 폴더 생성 + PRD/와이어프레임 배치
mkdir -p docs
# → PRD_카담_v3_4.md를 docs/PRD.md로 복사
# → wireframe_카담_v3_4.html을 docs/wireframe.html로 복사

# 4) scripts 폴더 생성
mkdir -p scripts
```

방법 2 — Cursor 파일 탐색기에서 직접 생성:

1. 왼쪽 파일 트리에서 프로젝트 루트 우클릭 → `New File`
2. 파일명: `.cursorrules` → 업로드한 dot_cursorrules.txt 내용 붙여넣기
3. `.cursor` 폴더 만들기 → 그 안에 `rules` 파일 생성 → cursor_rules.md 내용 붙여넣기
4. `docs` 폴더 만들기 → `PRD.md`, `wireframe.html` 파일 생성

결과 확인 — 프로젝트 루트 구조:

```
cadam/
├── .cursor/
│   └── rules              ← Cursor AI 규칙 (최신)
├── .cursorrules            ← Cursor AI 규칙 (레거시)
├── docs/
│   ├── PRD.md              ← 기획서 전문
│   └── wireframe.html      ← UI 와이어프레임
├── scripts/                ← (아직 비어있음)
├── src/
│   └── app/               ← Next.js가 만든 기본 구조
├── package.json
├── tailwind.config.ts
├── tsconfig.json
└── ... (기타 설정 파일들)
```

> **[사실]** PRD 13장 File Structure에서 이 구조가 정확히 명시되어 있습니다.
> cursor_rules에서도 "PRD: @docs/PRD.md", "와이어프레임: @docs/wireframe.html"로 참조 경로가 지정되어 있습니다.
> 이 파일들이 없으면 Cursor AI가 프로젝트 맥락을 이해할 수 없습니다.

🔵 Phase 3: 패키지(라이브러리) 설치

이것은 무엇인가?
다른 개발자들이 미리 만들어둔 코드 묶음을 가져다 쓰는 것입니다.
요리에 비유하면 "양념장을 직접 만들지 않고 마트에서 사오는 것"과 같습니다.

Cursor 터미널에서 한 번에 실행:

```
# 핵심 UI/상태관리
npm install zustand framer-motion

# Supabase (데이터베이스 + 인증)
npm install @supabase/supabase-js @supabase/ssr

# Rate Limiting (과도한 요청 차단)
npm install @upstash/ratelimit @upstash/redis

# 이메일 발송
npm install resend

# 입력값 검증
npm install zod

# XSS 방지 (Admin 페이지 보안)
npm install dompurify
npm install -D @types/dompurify
```

> **[사실]** 각 패키지가 필요한 이유:
> - zustand: PRD "State Management: Zustand + persist middleware"
> - framer-motion: PRD "전환: Framer Motion slide 좌↔우"
> - supabase: PRD "Supabase (PostgreSQL + Auth)"
> - upstash: PRD "Rate Limiting: Upstash Redis"
> - resend: PRD "이메일 알림 (Resend)"
> - zod: PRD "입력 검증: 서버사이드 zod 스키마"
> - dompurify: PRD "XSS 방지: DOMPurify (Admin 입력)"

설치 확인:

```
cat package.json
```

`dependencies` 안에 위 패키지들이 나열되어 있으면 성공.

🔵 Phase 4: Tailwind CSS 커스텀 색상 설정

4-1. tailwind.config.ts 수정

Cursor에서 `tailwind.config.ts` 파일을 엽니다 (왼쪽 파일 트리에서 클릭).

Cursor AI에게 지시하는 방법:
`Cmd+I` (Mac) 또는 `Ctrl+I` (Windows)를 눌러 인라인 편집 모드를 열고 다음과 같이 입력:

```
이 tailwind.config.ts에 다음 커스텀 색상을 추가해줘:
- primary: #1B3A5C
- accent: #2E86C1
- success: #27AE60
- warning: #F39C12
- danger: #E74C3C
- kakao: #FEE500
```

결과 예시 (AI가 생성할 코드):

```
// tailwind.config.ts
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./src/pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/components/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: '#1B3A5C',
        accent: '#2E86C1',
        success: '#27AE60',
        warning: '#F39C12',
        danger: '#E74C3C',
        kakao: '#FEE500',
      },
      fontFamily: {
        pretendard: ['Pretendard', '-apple-system', 'sans-serif'],
      },
    },
  },
  plugins: [],
};
export default config;
```

> **[사실]** cursor_rules: "색상은 Design System 변수 사용: primary(#1B3A5C)..."
> PRD 1장: "Colors: Primary #1B3A5C, Accent #2E86C1..."
> 이 색상을 Tailwind에 등록해야 `bg-primary`, `text-accent` 같은 클래스를 쓸 수 있습니다.

🔵 Phase 5: 외부 서비스 가입 & 환경변수 설정

5-1. Supabase 프로젝트 생성

이것은 무엇인가?
데이터베이스 + 사용자 인증을 제공하는 클라우드 서비스입니다.
고객 상담 신청 데이터, 가격 데이터, 관리자 로그인이 모두 여기에 저장됩니다.

단계:
1. https://supabase.com 접속 → Sign Up (GitHub 계정으로 가능)
2. "New Project" 클릭
3. 설정:
- **Name:** cadam
- **Database Password:** 강력한 비밀번호 입력 (메모해둘 것!)
- **Region:** Northeast Asia (Tokyo) — 한국에서 가장 빠름
4. 프로젝트 생성 완료 후 → Settings → API 탭에서:
- `Project URL` → 이것이 `NEXT_PUBLIC_SUPABASE_URL`
- `anon public` 키 → 이것이 `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `service_role` 키 → 이것이 `SUPABASE_SERVICE_ROLE_KEY` (절대 공개 금지!)

5-2. Upstash Redis 생성

이것은 무엇인가?
같은 사용자가 연속으로 폼을 제출하는 것을 막는 Rate Limiting용 서비스입니다.

단계:
1. https://upstash.com 접속 → Sign Up
2. "Create Database" → Redis 선택
3. 설정:
- **Name:** cadam-rate-limit
- **Region:** ap-northeast-1 (Tokyo)
4. 생성 후 REST API 탭에서:
- `UPSTASH_REDIS_REST_URL`
- `UPSTASH_REDIS_REST_TOKEN`

5-3. Resend 가입 (이메일 발송)

단계:

1. https://resend.com 접속 → Sign Up
2. API Keys → "Create API Key"
3. 발급된 키 → `RESEND_API_KEY`

> **[가정]** Resend 무료 티어(월 3,000건)로 MVP 기간에는 충분합니다.
> 도메인 인증 전에는 Resend 제공 이메일(@resend.dev)로 발송됩니다.

5-4. GA4 (Google Analytics) 설정

단계:

1. https://analytics.google.com 접속
2. "관리" → "만들기" → "속성" → 속성명: "카담"
3. 데이터 스트림 → "웹" → URL 입력 (임시로 localhost:3000)
4. 측정 ID (G-XXXXXXXXXX) → `NEXT_PUBLIC_GA_ID`

5-5. 환경변수 파일 생성 (.env.local)

이것은 무엇인가?

비밀번호, API 키 같은 민감 정보를 코드에 직접 쓰지 않고 별도 파일에 보관하는 것입니다.
이 파일은 Git에 올라가지 않아 보안이 유지됩니다.

Cursor에서 프로젝트 루트에 `.env.local` 파일 생성:

```
# ================================
# Supabase
# ================================
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJxxxxxxxxxxxx
SUPABASE_SERVICE_ROLE_KEY=eyJxxxxxxxxxxxx

# ================================
# Upstash Redis
# ================================
UPSTASH_REDIS_REST_URL=https://xxxxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AXxxxxxxxxxxxx

# ================================
# Resend (이메일)
# ================================
RESEND_API_KEY=re_xxxxxxxxxxxx
ADMIN_EMAIL=iori21y@gmail.com

# ================================
# GA4
# ================================
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX

# ================================
# ISR Revalidation Secret
# ================================
REVALIDATION_SECRET=여기에_긴_랜덤_문자열_입력

# ================================
# 카카오톡 채널 (개설 후 입력, 지금은 placeholder)
# ================================
NEXT_PUBLIC_KAKAO_CHANNEL_URL=https://pf.kakao.com/_placeholder

# ================================
# 운영 전화번호 (확정 후 입력, 지금은 placeholder)
# ================================
NEXT_PUBLIC_PHONE_NUMBER=02-0000-0000
```

REVALIDATION_SECRET 생성 방법:

```
# 터미널에서 실행 (Mac/Linux)
openssl rand -hex 32

# 결과 예: a1b2c3d4e5f6...  이 값을 .env.local에 붙여넣기
```

> **[사실]** PRD 14장에 환경변수 목록이 정확히 명시되어 있습니다.
> `NEXT_PUBLIC_` 접두사가 붙은 변수만 브라우저에서 접근 가능하고,
> 나머지는 서버에서만 접근 가능합니다 (보안 설계).

중요 확인: `.gitignore` 파일에 `.env.local`이 포함되어 있는지 확인!
(Next.js가 자동으로 추가하지만, 반드시 확인하세요)

🔵 Phase 6: 폴더 구조 생성

PRD에 명시된 폴더 구조를 미리 만들어두면 Cursor AI가 파일을 올바른 위치에 생성합니다.

Cursor 터미널에서 실행:

```
# src 하위 폴더 구조 생성
mkdir -p src/components/steps
mkdir -p src/components/cars
mkdir -p src/components/admin
mkdir -p src/constants
mkdir -p src/store
mkdir -p src/lib
mkdir -p src/hooks

# 페이지 폴더 구조 생성
mkdir -p src/app/info
mkdir -p src/app/result
mkdir -p src/app/cars
mkdir -p src/app/privacy
mkdir -p src/app/admin/login
mkdir -p src/app/admin/consultations
mkdir -p src/app/admin/promotions
mkdir -p src/app/admin/prices
mkdir -p src/app/api/consultation
mkdir -p src/app/api/price-range
mkdir -p src/app/api/promotions
mkdir -p src/app/api/revalidate

# 차량 이미지 폴더
mkdir -p public/cars
```

> **[사실]** PRD 13장 File Structure를 그대로 반영한 것입니다.
> cursor_rules: "본 PRD에 명시되지 않은 폴더 구조나 파일을 임의로 생성하지 말 것"
> 따라서 이 구조를 정확히 지켜야 합니다.

생성 후 프로젝트 구조 확인 (Cursor 왼쪽 파일 트리):

```
cadam/
├── .cursor/rules
├── .cursorrules
├── docs/
│   ├── PRD.md
│   └── wireframe.html
├── scripts/
├── public/
│   └── cars/              ← 차량 이미지 (나중에 추가)
├── src/
│   ├── app/
│   │   ├── layout.tsx     ← (Next.js 기본 생성)
│   │   ├── page.tsx       ← (Next.js 기본 생성)
│   │   ├── info/
│   │   ├── result/
│   │   ├── cars/
│   │   ├── privacy/
│   │   ├── admin/
│   │   │   ├── login/
│   │   │   ├── consultations/
│   │   │   ├── promotions/
│   │   │   └── prices/
│   │   └── api/
│   │       ├── consultation/
│   │       ├── price-range/
│   │       ├── promotions/
│   │       └── revalidate/
│   ├── components/
│   │   ├── steps/
│   │   ├── cars/
│   │   └── admin/
│   ├── constants/
│   ├── store/
│   ├── lib/
│   └── hooks/
├── .env.local
├── package.json
├── tailwind.config.ts
└── tsconfig.json
```

🔵 Phase 7: Cursor AI로 실제 코딩 시작하기

7-1. Cursor AI 사용법 — 세 가지 모드

모드	단축키	용도	설명
Inline Edit	`Cmd+I` / `Ctrl+I`	현재 열린 파일 수정	특정 파일 하나를 고칠 때
Chat	`Cmd+L` / `Ctrl+L`	질문 & 코드 생성	설명을 듣거나 코드 조각을 받을 때
Composer (Agent)	`Cmd+Shift+I` / `Ctrl+Shift+I`	**멀티파일 자동 생성**	여러 파일을 한 번에 만들 때 (핵심!)

> **[추론]** 카담 프로젝트처럼 여러 파일이 연결된 작업에서는
> **Composer(Agent) 모드**가 가장 효율적입니다.
> 한 번의 지시로 컴포넌트 + 스토어 + 유틸리티를 동시에 생성할 수 있습니다.

7-2. @ 참조 기능 (컨텍스트 제공)

Cursor AI에게 참고할 파일을 알려주는 핵심 기능입니다.

```
@docs/PRD.md            ← PRD 전체를 AI에게 보여줌
@docs/wireframe.html    ← 와이어프레임을 AI에게 보여줌
@src/constants/vehicles.ts  ← 차량 데이터를 AI에게 보여줌
```

사용 예시 (Composer에서):

```
@docs/PRD.md 의 4장을 참고해서
src/constants/vehicles.ts 파일을 만들어줘.
Vehicle 인터페이스, Brand/Category 타입, VEHICLE_LIST 45종 전체,
유틸리티 함수 4개를 포함해야 해.
```

7-3. 첫 번째 작업 — 차량 데이터 상수 파일 생성

왜 이것부터 하는가?
- 이 파일은 다른 파일에 의존하지 않음 (독립적)
- 거의 모든 페이지에서 이 데이터를 참조함
- AI가 정확하게 생성했는지 검증하기 쉬움 (45종 카운트만 하면 됨)

Composer(Agent) 모드 열기: `Cmd+Shift+I`

프롬프트:

```
@docs/PRD.md 의 "4. Vehicle Data" 섹션을 참고해서
src/constants/vehicles.ts 파일을 생성해줘.

포함해야 할 것:
1. Category, Brand 타입 (export type)
2. Vehicle 인터페이스 (export interface)
3. VEHICLE_LIST 상수 — 현대 19종, 기아 19종, 제네시스 7종 = 총 45종
4. 유틸리티 함수: getVehicleBySlug, getVehiclesByBrand, getVehiclesByCategory, getAllSlugs

PRD에 있는 데이터를 그대로 사용할 것.
any 타입 금지, interface 사용.
```

AI가 코드를 생성하면:

1. "Accept" 버튼을 클릭 (코드가 파일에 반영됨)
2. 생성된 파일을 열어서 차량이 45종인지 확인

7-4. 두 번째 작업 — Zustand 스토어 생성

프롬프트:

```
@docs/PRD.md 의 "5.1. State Management"와 "5.2. Step Flow 상세"를 참고해서
src/store/quoteStore.ts 파일을 생성해줘.

Zustand + persist 미들웨어를 사용하고,
다음 상태를 포함해야 해:
- currentStep (number)
- selectionPath ('car' | 'budget' | null)
- carBrand, carModel, trim
- contractMonths, annualKm
- deposit, prepaymentPct
- monthlyBudget
- name, phone, privacyAgreed
- 각 상태를 업데이트하는 setter 함수들
- resetAll 함수 (스토어 초기화)

localStorage 직접 접근 금지, persist를 통해서만.
any 타입 금지.
```

7-5. 세 번째 작업 — 유틸리티 라이브러리

프롬프트:

```
@docs/PRD.md 를 참고해서 다음 파일들을 한 번에 생성해줘:

1. src/lib/supabase.ts — Supabase 클라이언트 (환경변수 사용)
2. src/lib/leadScore.ts — PRD 3.4장의 리드 점수 계산 로직 그대로 구현
   (서버사이드 전용 주석 포함)
3. src/lib/rateLimit.ts — PRD 8.2장의 Upstash Redis Rate Limiter 그대로 구현
4. src/lib/utils.ts — 전화번호 하이픈 제거, 포맷팅, IP SHA-256 해시 함수
5. src/lib/gtag.ts — GA4 이벤트 트래킹 유틸리티
6. src/lib/notification.ts — Resend 이메일 발송 (PRD 8.5장)
7. src/lib/kakao.ts — Phase 2 placeholder (함수 시그니처 + 주석만)

모든 파일은 TypeScript, any 타입 금지.
kakao_sent 관련 로직 작성 금지.
```

7-6. 네 번째 작업 — 훅(Hooks) 생성

프롬프트:

```
@docs/PRD.md 를 참고해서:

1. src/hooks/useHydrated.ts — SSR Hydration mismatch 방지 훅
   (Zustand persist가 클라이언트에서만 작동하므로 필요)
2. src/hooks/useLeaveIntent.ts — 이탈 감지 훅
   Desktop: 마우스 Y ≤ 0
   Mobile: popstate 이벤트
   조건: Step 2 이상, 세션당 1회만 트리거
```

🔵 Phase 8: 데이터베이스 설정 (Supabase)

8-1. 테이블 생성

Supabase Dashboard(https://supabase.com/dashboard)에 로그인 후:

1. 왼쪽 메뉴 → **SQL Editor** 클릭
2. "New Query" 클릭
3. 아래 SQL을 Cursor AI에게 생성시키거나, 직접 붙여넣기

Cursor Composer에서 SQL 생성 요청:

```
@docs/PRD.md 의 "3. Database & Security" 섹션을 참고해서
Supabase에서 실행할 SQL 스크립트를 생성해줘.

포함할 것:
1. consultations 테이블 (22개 컬럼 전부)
2. price_ranges 테이블 (conditions jsonb 포함)
3. promotions 테이블
4. notification_log 테이블
5. 3.2장의 RLS 정책 전부
6. 3.3장의 인덱스 7개 전부

파일 위치: scripts/seed.ts가 아니라 별도 SQL 파일로.
```

생성된 SQL을 Supabase SQL Editor에 붙여넣고 "Run" 클릭.

8-2. 관리자 계정 생성

Supabase Dashboard → **Authentication** → **Users** 탭:
1. "Add User" → "Create New User"
2. Email: `iori21y@gmail.com`
3. Password: 원하는 강력한 비밀번호 입력
4. "Auto Confirm User" 체크
5. "Create User" 클릭

🔵 Phase 9: 첫 화면 만들기 (Step 플로우 시작)

여기서부터 본격적인 UI 개발입니다.

9-1. 루트 레이아웃 수정

프롬프트:

```
@docs/PRD.md 1장과 @docs/wireframe.html 을 참고해서
src/app/layout.tsx를 수정해줘.

- Pretendard 웹폰트 (CDN, font-display: swap, preload)
- GA4 스크립트 (next/script, NEXT_PUBLIC_GA_ID)
- 메타데이터: title "카담(CADAM) | 장기렌터카 맞춤 견적"
- body에 max-width 1024px 중앙 정렬 컨테이너
- 모바일 퍼스트: min-width 360px

Tailwind CSS만 사용, 인라인 style 금지.
```

9-2. Step 공통 레이아웃 + ProgressBar

프롬프트:

```
@docs/PRD.md 5.2장 "공통 레이아웃"과 @docs/wireframe.html 을 참고해서:

1. src/components/steps/StepLayout.tsx
   - 100vh 풀스크린, 배경 #FFFFFF
   - 상단: ProgressBar
   - 중앙: children (질문 + 선택지)
   - 하단: 이전/다음 버튼 (고정)
   - Framer Motion slide 좌↔우 전환
   - 'use client' 필수

2. src/components/steps/ProgressBar.tsx
   - currentStep / totalSteps (6)
   - accent 색상 (#2E86C1) 프로그레스 바

와이어프레임의 레이아웃과 간격을 최대한 따르되,
스타일은 반드시 Tailwind CSS 클래스로 구현.
인라인 style이나 CSS 변수 사용 금지.
```

9-3. Step 1 ~ Step 6 컴포넌트 (하나씩 순서대로!)

각 Step을 **반드시 하나씩** 만드세요. 한 번에 전부 요청하면 AI가 세부 스펙을 누락합니다.
하나를 만들고 → `npm run dev` 로 화면 확인 → 문제 없으면 다음으로 넘어가세요.

> **[사실]** PRD 5.2장에 각 Step의 질문 텍스트, 선택지, 동작이 정확히 명시되어 있습니다.
> 아래 프롬프트는 PRD 내용을 그대로 반영한 것입니다.
> **[추론]** 와이어프레임(@docs/wireframe.html)을 함께 참조시키면
> AI가 카드 모양, 간격, 선택 상태 표시까지 맞출 수 있습니다.

---

① Step 1 — 선택 분기 (Step1Branch.tsx)

이 Step의 역할: 사용자가 "차종 기준"으로 갈지 "예산 기준"으로 갈지 선택하는 첫 화면.
이 선택에 따라 Step 2에서 보여줄 화면이 달라집니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 1: 선택 분기"와
@docs/wireframe.html 의 Step 1 화면을 참고해서
src/components/steps/Step1Branch.tsx를 만들어줘.

■ 화면 구성:
- 질문: "당신에게 더 중요한 선택은 무엇인가요?"
- 선택지 2개 (세로 배치, 큰 카드형 버튼):
  1. 🚗 "차종 먼저 선택할게요" → Zustand에 selectionPath: 'car' 저장
  2. 💰 "월 예산 먼저 정할게요" → Zustand에 selectionPath: 'budget' 저장

■ 동작:
- 카드를 탭하면 선택 상태(테두리 accent 색상 + 배경 accent-light)로 변하고
  0.3초 후 자동으로 다음 Step(currentStep +1)으로 이동
- Zustand 스토어 @src/store/quoteStore.ts 의
  setSelectionPath()와 setCurrentStep() 사용

■ 스타일 규칙:
- 카드: border-radius 12px, border 2px, 선택 시 border-color accent
- 이모지는 font-size 24px 크기
- Tailwind CSS만 사용, 인라인 style 금지
- 와이어프레임의 .opt-card 스타일을 Tailwind로 변환

■ 필수:
- 'use client' 지시어 맨 위에
- any 타입 금지, props는 interface로 정의
- StepLayout은 이 컴포넌트 안에서 import하지 않음 (부모에서 감싸줌)
```

---

② Step 2A — 차종 선택 (Step2Car.tsx)

이 Step의 역할: 브랜드 → 카테고리 → 차종 → 트림 순서로 좁혀가며 원하는 차를 선택.
이 컴포넌트가 전체 프로젝트에서 가장 복잡합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 2A: 차종 선택"과
@docs/wireframe.html 의 Step 2 차종 선택 화면을 참고해서
src/components/steps/Step2Car.tsx를 만들어줘.

■ 화면 구성 (위에서 아래 순서):

1) 브랜드 탭 (상단 고정):
   - 3개 탭: "현대" | "기아" | "제네시스"
   - 활성 탭: 하단 보더 accent 색상 + 텍스트 accent
   - 비활성 탭: 텍스트 gray
   - 기본 선택: "현대"

2) 카테고리 서브 필터 (탭 아래 가로 칩):
   - "전체" | "세단" | "SUV" | "EV" (필터 칩, 가로 스크롤)
   - 활성 칩: accent 배경 + 흰 텍스트
   - 비활성 칩: gray 보더 + gray 텍스트
   - "전체"는 해당 브랜드의 모든 차종 표시
   - 다목적/트럭 카테고리 차종은 "전체"에서만 노출

3) 차종 카드 그리드 (스크롤 영역):
   - 2열 그리드 (grid-cols-2)
   - 각 카드: 차종명(볼드) + 세그먼트 태그(작은 회색 텍스트)
   - 카드 border-radius 12px, border 2px solid gray
   - 선택 시: border accent + 배경 accent-light

4) 트림 선택 (차종 카드 탭 시 나타남):
   - 모바일: 화면 하단에서 올라오는 바텀시트(슬라이드업)
   - 데스크톱(md 이상): 선택한 카드 아래 인라인 확장
   - 트림 목록은 해당 차종의 trims 배열에서 가져옴
   - 각 트림을 라디오 버튼 또는 카드로 표시
   - 트림 선택 완료 시 → Zustand에 carBrand, carModel, trim 저장
     → 자동으로 Step 3으로 이동

■ 데이터 소스:
- 반드시 @src/constants/vehicles.ts 의 VEHICLE_LIST 사용
- getVehiclesByBrand()로 브랜드별 필터
- 카테고리 필터는 프론트엔드에서 .filter(v => v.category === selected)
- 와이어프레임에 하드코딩된 차량 데이터(26종)는 무시할 것

■ Zustand 연결:
- @src/store/quoteStore.ts 의
  setCarBrand(), setCarModel(), setTrim(), setCurrentStep() 사용

■ 스타일 규칙:
- Tailwind CSS만 사용, 인라인 style 금지
- 와이어프레임의 .tabs, .sub-tabs, .grid-2, .grid-card 스타일을 Tailwind로 변환
- 바텀시트는 Framer Motion animate로 슬라이드업 구현

■ 필수:
- 'use client' 필수
```

```
- any 타입 금지
- 빈 결과 처리: 해당 브랜드+카테고리에 차종이 없으면 "해당 차종이 없습니다" 표시
```

---

③ Step 2B — 예산 선택 (Step2Budget.tsx)

이 Step의 역할: Step 1에서 "예산 먼저"를 선택한 사용자에게 월 예산 범위를 물어보는 화면.
차종 선택보다 훨씬 간단합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 2B: 예산 선택"과
@docs/wireframe.html 의 예산 선택 화면을 참고해서
src/components/steps/Step2Budget.tsx를 만들어줘.

■ 화면 구성:
- 질문: "월 예산은 어느 정도 생각하고 계세요?"
- 선택지 5개 (세로 배치, 카드형 버튼):
  1. 💰 "30만원 이하" → monthlyBudget: 300000
  2. 💰 "30~50만원" → monthlyBudget: 500000
  3. 💰 "50~70만원" → monthlyBudget: 700000
  4. 💰 "70~100만원" → monthlyBudget: 1000000
  5. 💰 "100만원 이상" → monthlyBudget: 1500000

■ 동작:
- 카드 탭 시 선택 상태 표시(accent 테두리 + accent-light 배경)
- 0.3초 후 자동으로 Step 3으로 이동
- Zustand에 monthlyBudget 값 저장

■ Zustand 연결:
- @src/store/quoteStore.ts 의 setMonthlyBudget(), setCurrentStep() 사용

■ 스타일 규칙:
- Step1Branch와 동일한 카드 스타일 (일관성)
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
```

---

④ Step 3 — 계약 기간 (Step3Period.tsx)

이 Step의 역할: 장기렌트 계약 기간을 선택. 48개월이 가장 인기 있어서 "인기" 뱃지를 표시.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 3: 계약 기간"과
@docs/wireframe.html 의 Step 3 화면을 참고해서
src/components/steps/Step3Period.tsx를 만들어줘.

■ 화면 구성:
- 질문: "계약 기간을 선택해 주세요"
- 부가 설명: "기간이 길수록 월 납부금이 낮아집니다"
- 선택지 3개 (세로 배치, 카드형 버튼):
  1. 📅 "36개월" / 부제: "3년" → contractMonths: 36
  2. 📅 "48개월" / 부제: "4년" → contractMonths: 48
     ★ 이 카드의 오른쪽 상단에 "인기" 뱃지 표시
       (뱃지: warning 배경 #F39C12, 흰 텍스트, border-radius 10px, font-size 11px)
  3. 📅 "60개월" / 부제: "5년" → contractMonths: 60

■ 동작:
- 카드 탭 시 선택 상태 표시
- 0.3초 후 자동으로 Step 4로 이동
- Zustand에 contractMonths 저장

■ Zustand 연결:
- @src/store/quoteStore.ts 의 setContractMonths(), setCurrentStep() 사용

■ 스타일 규칙:
- 카드 스타일은 Step 1과 동일 패턴
- "인기" 뱃지: position absolute, top -8px, right 12px
  (와이어프레임의 .badge 클래스 참고)
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
```

---

⑤ Step 4 — 연간 주행거리 (Step4Mileage.tsx)

이 Step의 역할: 연간 예상 주행거리를 선택. 각 선택지에 "어떤 용도인지" 부가 설명이 붙어서 사용자가 쉽게 이해할 수 있게 합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 4: 주행거리"와
@docs/wireframe.html 의 Step 4 화면을 참고해서
src/components/steps/Step4Mileage.tsx를 만들어줘.

■ 화면 구성:
- 질문: "연간 주행거리를 선택해 주세요"
- 부가 설명: "실제 주행거리를 초과하면 추가 요금이 발생할 수 있습니다"
- 선택지 4개 (세로 배치, 카드형 버튼):
  1. 🚗 "연 1만 km" / 부제: "주말 위주" → annualKm: 10000
  2. 🚗 "연 2만 km" / 부제: "출퇴근 기본" → annualKm: 20000
     ★ "인기" 뱃지 표시 (Step 3의 뱃지와 동일 스타일)
  3. 🚗 "연 3만 km" / 부제: "업무 겸용" → annualKm: 30000
  4. 🚗 "연 4만 km+" / 부제: "장거리" → annualKm: 40000

■ 동작:
- 카드 탭 시 선택 상태 표시
- 0.3초 후 자동으로 Step 5로 이동
- Zustand에 annualKm 저장

■ Zustand 연결:
- @src/store/quoteStore.ts 의 setAnnualKm(), setCurrentStep() 사용

■ 스타일 규칙:
- 카드의 main label은 font-weight 600, 16px
- 부제(sub)는 font-size 13px, gray-500 색상
- "인기" 뱃지: Step 3과 동일 (warning 배경, 흰 텍스트)
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
```

---

⑥ Step 5 — 보증금 + 선납금 (Step5Payment.tsx)

이 Step의 역할: 보증금과 선납금 비율을 한 화면에서 선택.
다른 Step과 달리 두 가지를 동시에 선택해야 하므로 "자동 다음 이동"이 아니라
하단의 "다음" 버튼을 눌러야 이동합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 5: 보증금 + 선납금"과
@docs/wireframe.html 의 Step 5 화면을 참고해서
src/components/steps/Step5Payment.tsx를 만들어줘.

■ 화면 구성 (같은 화면 내 스크롤):

[섹션 1 — 보증금]
- 소제목: "보증금을 선택해 주세요"
- 부가 설명: "보증금이 높을수록 월 납부금이 낮아집니다"
- 선택지 4개 (가로 2열 그리드):
  1. "0원" (없음) → deposit: 0
  2. "100만원" → deposit: 1000000
  3. "200만원" → deposit: 2000000
  4. "300만원+" → deposit: 3000000

[섹션 2 — 선납금] (보증금 아래에 이어서)
- 소제목: "선납금 비율을 선택해 주세요"
- 부가 설명: "차량가의 일부를 선납하면 월 납부금이 낮아집니다"
- 선택지 4개 (가로 2열 그리드):
  1. "0%" (없음) → prepaymentPct: 0
  2. "10%" → prepaymentPct: 10
  3. "20%" → prepaymentPct: 20
  4. "30%" → prepaymentPct: 30

■ 동작:
- 보증금과 선납금 각각 하나씩 선택 (둘 다 선택해야 함)
- 선택된 카드: accent 테두리 + accent-light 배경
- 이 Step은 자동 이동하지 않음!
  → 두 가지 모두 선택한 후 하단 "다음" 버튼(StepLayout에서 제공)으로 이동
- Zustand에 deposit, prepaymentPct 저장

■ Zustand 연결:
- @src/store/quoteStore.ts 의 setDeposit(), setPrepaymentPct() 사용
- 두 값이 모두 선택되었는지 확인하는 로직 필요
  (부모 StepLayout의 "다음" 버튼 활성/비활성 제어를 위해
   isStepComplete 같은 boolean을 props나 콜백으로 전달)

■ 스타일 규칙:
- 2열 그리드: grid-cols-2, gap 10px
- 각 카드: border-radius 12px, border 2px, text-align center
- 두 섹션 사이에 구분선 또는 충분한 여백(margin-top 24px 이상)
- 와이어프레임의 .grid-2, .grid-card 스타일을 Tailwind로 변환
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
```

⑦ Step 6 — 연락처 + 개인정보 동의 (Step6Contact.tsx)

이 Step의 역할: 최종 단계. 사용자 이름과 연락처를 입력받고,
개인정보 수집 동의를 받은 뒤 상담 신청을 제출합니다.
이 Step이 제출되면 /api/consultation API가 호출됩니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.2장 "Step 6: 연락처 + 개인정보 동의"와
@docs/wireframe.html 의 Step 6 화면을 참고해서
src/components/steps/Step6Contact.tsx를 만들어줘.

■ 화면 구성:

[필드 1 — 이름]
- 라벨: "이름" (필수)
- <input type="text" placeholder="홍길동" />
- 미입력 시 하단에 빨간 인라인 텍스트: "이름을 입력해 주세요"

[필드 2 — 연락처]
- 라벨: "연락처" (필수)
- <input type="tel" placeholder="010-0000-0000" />
- onChange에서 Auto-formatting 구현:
  숫자만 추출 → "010-XXXX-XXXX" 형태로 자동 하이픈 삽입
  (예: 01012345678 입력 → 화면에 010-1234-5678 표시)
- 제출 시에는 하이픈 제거한 숫자만 서버로 전송
- 정규식 검증: /^01([016789])-?(\d{3,4})-?(\d{4})$/
- 불일치 시 하단에 빨간 인라인 텍스트: "올바른 연락처를 입력해 주세요"

[체크박스 — 개인정보 동의]
- 체크박스 + "개인정보 수집 및 이용에 동의합니다"
- "[전문보기]" 링크 → /privacy 새 탭으로 열기 (target="_blank")
- 체크박스 accent-color: accent (#2E86C1)

[제출 버튼]
- 텍스트: "무료 견적 받기"
- 스타일: 전체 폭(w-full), accent 배경, 흰 텍스트, border-radius 8px, 최소 높이 48px
- 비활성 조건: 이름 미입력 OR 연락처 정규식 불일치 OR 개인정보 미동의
  → 비활성 시: gray-300 배경, gray-500 텍스트, cursor-default
- 활성 시: 클릭하면 /api/consultation POST 호출

[로딩 상태]
- 제출 중: 버튼 내부에 스피너 아이콘 + "견적 요청 중..." 텍스트
- 버튼 disabled 처리 (중복 클릭 방지)

■ API 호출 로직:
- POST /api/consultation 호출
- body: Zustand 스토어의 모든 값 (selectionPath, carBrand, carModel, trim,
  contractMonths, annualKm, deposit, prepaymentPct, monthlyBudget,
  name, phone(하이픈 제거), privacyAgreed: true)
- 성공 시 (200): router.push('/result')로 결과 페이지 이동
- 실패 시 (429): Toast로 "과도한 요청입니다. 1분 후 다시 시도해 주세요." 표시
- 실패 시 (409 중복): Toast로 "이미 접수된 신청이 있습니다. 상담사가 곧 연락드립니다." 표시
- 실패 시 (기타): Toast로 "네트워크 오류가 발생했습니다. 다시 시도해 주세요." 표시

■ Zustand 연결:
- @src/store/quoteStore.ts 의 모든 상태 값 읽기
- setName(), setPhone(), setPrivacyAgreed() 사용
```

```
■ 전화번호 포맷팅 유틸:
- @src/lib/utils.ts 의 전화번호 관련 함수 사용
  (아직 없으면 이 컴포넌트 내부에 헬퍼 함수로 임시 구현해도 OK)

■ 스타일 규칙:
- 입력 필드: border-radius 8px, border 1px gray-300, padding 14px 16px, font-size 16px
- focus 시: border-color accent
- 에러 텍스트: font-size 12px, text-danger(#E74C3C), margin-top 4px
- 와이어프레임의 .form-group, .form-input, .checkbox-row 스타일을 Tailwind로 변환
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
- 폼 유효성 검사는 프론트엔드에서 (UX용), 서버사이드에서도 zod로 재검증 (보안용)
- leadScore 계산 로직을 이 컴포넌트에 절대 넣지 말 것 (서버사이드 전용)
```

📋 Step 생성 순서 요약 & 검증 포인트

순서	파일명	생성 후 확인할 것
1	Step1Branch.tsx	카드 2개 표시, 탭 시 선택 상태 변경, Step 2로 이동
2	Step2Car.tsx	브랜드탭 전환, 카테고리 필터 작동, 차종 45종 중 해당 차종만 표시, 트림 시트 열림
3	Step2Budget.tsx	예산 카드 5개 표시, 탭 시 선택 후 Step 3 이동
4	Step3Period.tsx	3개 카드 + 48개월에 "인기" 뱃지, 탭 시 Step 4 이동
5	Step4Mileage.tsx	4개 카드 + 2만km에 "인기" 뱃지, 탭 시 Step 5 이동
6	Step5Payment.tsx	보증금 4개 + 선납금 4개 동시 선택, 둘 다 선택해야 "다음" 활성화
7	Step6Contact.tsx	이름/연락처 입력, 자동 하이픈, 유효성 검사, 동의 체크, 제출 + 로딩

각 Step 생성 후 터미널에서:

```
npm run dev
# http://localhost:3000 에서 해당 Step까지 진행해보며 UI 확인
# 에러가 있으면 Cursor Chat(Cmd+L)에 에러 메시지 붙여넣고 수정 요청
```

> **[추론]** Step2Car.tsx가 가장 복잡하므로 가장 많은 수정이 필요할 것입니다.
> AI가 처음 생성한 결과에서 트림 바텀시트, 카테고리 필터, 빈 결과 처리 등이
> 누락될 수 있으니 화면을 꼼꼼히 확인하고 부족한 부분을 추가 요청하세요.

9-4. 메인 페이지 조립

프롬프트:

```
src/app/page.tsx를 수정해서
Step 1~6을 StepLayout 안에서 currentStep에 따라 조건부 렌더링해줘.

@src/store/quoteStore.ts의 currentStep 사용.
@src/components/steps/ 하위 컴포넌트들을 import.
@src/hooks/useHydrated.ts로 Hydration 처리.

selectionPath에 따라 Step 2A(차종) 또는 Step 2B(예산) 분기.
'use client' 필수.
```

🔵 Phase 10: API Routes 구현

Phase 9까지 완료하면 Step 1~6 화면이 보이지만, "무료 견적 받기" 버튼을 눌러도
서버에 데이터가 저장되지 않습니다. 이제 백엔드를 만듭니다.

> **[사실]** PRD 8장에 모든 API의 처리 흐름이 명시되어 있습니다.
> **[추론]** API는 서로 독립적이므로 하나씩 만들고 테스트하는 것이 안전합니다.

10-1. POST /api/consultation (상담 신청 — 가장 핵심)

이 API의 역할: Step 6에서 "무료 견적 받기"를 누르면 호출됩니다.
고객 정보를 DB에 저장하고, 관리자에게 이메일을 보내고, 예상 견적을 응답합니다.
PRD 8.3장의 10단계를 정확히 구현해야 합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 8.3장 "POST /api/consultation" 처리 흐름을 참고해서
src/app/api/consultation/route.ts 를 만들어줘.

■ 10단계 처리 흐름 (반드시 이 순서대로):

1단계 - Rate Limit 체크:
   - @src/lib/rateLimit.ts 의 rateLimiter.limit(ip) 호출
   - IP는 request.headers.get('x-forwarded-for') 또는
     request.headers.get('x-real-ip') 에서 추출
   - 실패 시 429 응답: { error: "Too many requests" }

2단계 - zod 입력 검증:
   - zod 스키마 정의:
     name: string, min 1, max 50
     phone: string, 정규식 /^01([016789])\d{3,4}\d{4}$/
     privacyAgreed: boolean (true만 허용)
     selectionPath: 'car' | 'budget'
     carBrand: string nullable
     carModel: string nullable
     trim: string nullable
     contractMonths: number nullable (36|48|60)
     annualKm: number nullable (10000|20000|30000|40000)
     deposit: number nullable
     prepaymentPct: number nullable
     monthlyBudget: number nullable
   - 검증 실패 시 400 응답

3단계 - IP 해시 생성:
   - crypto.createHash('sha256').update(ip).digest('hex')

4단계 - 중복 체크:
   - Supabase에서 24시간 내 동일 ip_hash 존재 여부 확인
   - 존재 시 409 응답: { warning: "duplicate", message: "이미 접수된 신청이 있습니다" }
   (INSERT는 하지 않음)

5단계 - Lead Score 계산:
   - @src/lib/leadScore.ts 의 calculateLeadScore() 서버사이드 호출
   - input: { stepCompleted, carModel, contractMonths, deposit, monthlyBudget,
inflowPage }

6단계 - Cookie 파싱:
   - request.cookies에서 device_type, utm_source, referrer, inflow_page 읽기

7단계 - Supabase INSERT:
   - consultations 테이블에 모든 필드 INSERT
   - supabase는 service_role key 사용 (서버사이드이므로)
   - status 기본값: 'pending'

8단계 - 가격 매칭:
   - selectionPath === 'car'이고 carBrand, carModel, contractMonths, annualKm가 있으면
     price_ranges 테이블에서 매칭 조회
   - 매칭 성공 시: estimated_min, estimated_max를 consultations UPDATE
```

```
   - 매칭 실패 또는 budget 경로: null 유지

9단계 - 이메일 발송:
   - @src/lib/notification.ts 사용
   - To: process.env.ADMIN_EMAIL
   - Subject: "[새 상담] {이름} - {차종} (리드: {lead_score}점)"
   - Body: HTML 테이블 (고객 정보 + 선택 내역 + 리드 등급)
   - notification_log 테이블에 발송 결과 INSERT
   - 이메일 실패해도 상담 신청 자체는 성공 처리 (try-catch)

10단계 - 응답:
   - 200: { success: true, estimatedMin, estimatedMax }

■ 보안 규칙:
- Supabase service_role key 사용 (SUPABASE_SERVICE_ROLE_KEY)
- 이 파일에 'use client' 절대 넣지 말 것 (서버 전용)
- any 타입 금지, zod 스키마로 타입 추론

■ 에러 핸들링:
- 각 단계에서 에러 발생 시 적절한 HTTP 상태코드 반환
- 전체를 try-catch로 감싸서 500 에러 방지
```

10-2. GET /api/price-range (견적 조회)

Cursor Composer 프롬프트:

```
@docs/PRD.md 8.4장을 참고해서
src/app/api/price-range/route.ts 를 만들어줘.

■ 쿼리 파라미터:
- brand (string, 필수): "현대" | "기아" | "제네시스"
- model (string, 필수): 차종명
- months (number, 필수): 36 | 48 | 60
- km (number, 필수): 10000 | 20000 | 30000 | 40000

■ 처리:
- Supabase price_ranges 테이블에서 조건 매칭 (is_active=true)
- 매칭 성공: { minMonthly, maxMonthly, conditions }
- 매칭 실패: { minMonthly: null, maxMonthly: null, message: "상담 문의" }

■ 규칙:
- Supabase anon key 사용 가능 (공개 데이터)
- any 타입 금지
- 'use client' 절대 넣지 말 것
```

10-3. GET /api/promotions (프로모션 조회)

Cursor Composer 프롬프트:

```
@docs/PRD.md 7장을 참고해서
src/app/api/promotions/route.ts 를 만들어줘.

■ 처리:
- Supabase promotions 테이블에서 is_active=true인 항목 조회
- display_order 오름차순 정렬
- 현재 날짜가 start_date~end_date 범위 내인 것만 반환

■ 응답:
- { promotions: [...] }

■ 규칙:
- Supabase anon key 사용 가능
- any 타입 금지
```

10-4. POST /api/revalidate (ISR 재생성)

Cursor Composer 프롬프트:

```
@docs/PRD.md 6.1장의 revalidate 코드를 참고해서
src/app/api/revalidate/route.ts 를 만들어줘.

■ 처리:
- body에서 slug, secret 추출
- secret이 process.env.REVALIDATION_SECRET과 불일치 시 401 반환
- revalidatePath('/cars/' + slug) 호출
- { revalidated: true } 응답

■ PRD에 코드가 이미 있으므로 그대로 구현하되:
- zod로 입력 검증 추가
- any 타입 금지
```

Phase 10 완료 후 확인:

```
# 터미널에서 API 테스트
curl -X POST http://localhost:3000/api/consultation \
  -H "Content-Type: application/json" \
  -d '{"name":"테스트","phone":"01012345678","privacyAgreed":true,"selectionPath":"car"}'

# 200 응답이 오면 성공. Supabase Dashboard에서 데이터 확인.
```

🔵 Phase 11: 견적 결과 페이지

이 페이지의 역할: Step 6에서 폼 제출 성공 후 리다이렉트되는 화면입니다.
고객에게 예상 견적을 보여주고, 카카오톡/전화 상담으로 유도합니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.3장 "견적 결과 화면"과
@docs/wireframe.html 의 결과 화면을 참고해서
src/app/result/page.tsx를 만들어줘.

■ 화면 분기 (Zustand 스토어의 selectionPath에 따라):

[budget 경로] (selectionPath === 'budget'):
- API 조회 생략
- 상단 아이콘: ✅ 체크마크 (success 배경 원형)
- 메시지: "고객님 예산에 맞는 최적의 차량을 찾고 있습니다"
- 부가 텍스트: "전문 상담사가 맞춤 견적을 준비하여 연락드리겠습니다"

[car 경로] (selectionPath === 'car'):
- Zustand에 저장된 estimatedMin, estimatedMax 값 사용
  (Step 6 제출 API 응답에서 받아서 스토어에 저장해둔 값)
- 데이터 있을 때:
  - 상단 아이콘: ✅
  - 가격 카드: 그라디언트 배경(primary→accent), "예상 월 납부금" 라벨
    금액: "월 {min}~{max}만원" (큰 폰트, 볼드)
  - 요약 카드: 선택한 차종/기간/거리/보증금/선납금 한눈에 표시
- 데이터 없을 때:
  - "상담사가 정확한 견적을 안내해 드립니다"

■ CTA 버튼 (하단 고정):
1. "카카오톡으로 상담하기" (kakao 노란색 배경, 전체 폭)
   → process.env.NEXT_PUBLIC_KAKAO_CHANNEL_URL 새 탭
2. "전화 상담 요청" (accent 아웃라인, 전체 폭)
   → tel:process.env.NEXT_PUBLIC_PHONE_NUMBER
3. "다른 차량도 견적 받아보기" (텍스트 링크)
   → Zustand 스토어 resetAll() 후 router.push('/')

■ Zustand 연결:
- @src/store/quoteStore.ts 에서 모든 선택값 + estimatedMin/Max 읽기
- resetAll()로 스토어 초기화

■ 스타일:
- 와이어프레임의 .result-icon, .price-card, .summary-card 스타일을 Tailwind로 변환
- 가격 카드 그라디언트: bg-gradient-to-br from-primary to-accent
- Tailwind CSS만 사용, 인라인 style 금지

■ 필수:
- 'use client' 필수
- any 타입 금지
- 스토어가 비어있는 경우(직접 URL 접근) → '/' 로 리다이렉트
```

🔵 Phase 12: Toast + 이탈 방지 모달

12-1. Toast 컴포넌트

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.5장 "에러 처리 & Toast"와
@docs/wireframe.html 을 참고해서
src/components/Toast.tsx를 만들어줘.

■ 기능:
- 화면 하단 중앙에 표시 (모바일 safe-area 고려, bottom: 80px)
- 애니메이션: Framer Motion으로 하단에서 슬라이드업 → 3초 유지 → 페이드아웃
- 색상 타입:
  - error: danger 배경(#E74C3C) + 흰 텍스트
  - success: success 배경(#27AE60) + 흰 텍스트
  - warning: warning 배경(#F39C12) + 흰 텍스트

■ 사용 방식 (다른 컴포넌트에서 호출하는 인터페이스):
- 전역 상태로 관리하거나 Context로 관리
  (예: useToast() 훅 → { showToast(message, type) })
- 또는 Zustand 스토어에 toast 상태 추가

■ PRD에 명시된 Toast 메시지:
- 네트워크 에러: "네트워크 오류가 발생했습니다. 다시 시도해 주세요." (error)
- 429 Rate Limit: "과도한 요청입니다. 1분 후 다시 시도해 주세요." (error)
- 중복 IP: "이미 접수된 신청이 있습니다. 상담사가 곧 연락드립니다." (warning)

■ 스타일:
- border-radius 8px, padding 12px 20px, shadow
- z-index 최상위 (모달보다는 아래)
- Tailwind CSS만 사용

■ 필수:
- 'use client' 필수
- any 타입 금지
```

12-2. 이탈 방지 모달

Cursor Composer 프롬프트:

```
@docs/PRD.md 5.4장 "이탈 방지 시스템"과
@docs/wireframe.html 의 이탈 모달 화면을 참고해서
src/components/LeaveModal.tsx를 만들어줘.

■ 트리거 조건 (@src/hooks/useLeaveIntent.ts 사용):
- Desktop: 마우스 커서 Y좌표 ≤ 0 (브라우저 상단으로 벗어남)
- Mobile: popstate 이벤트 (뒤로가기 시도)
- 조건: currentStep이 2 이상일 때만
- 제한: 세션당 1회만 표시

■ 모달 내용:
- 아이콘: ⚠️
- 제목: "잠깐, 아직 견적이 완성되지 않았어요!"
- 설명: "지금까지 선택한 내용이 사라질 수 있습니다.
        견적을 완성하고 최저가를 확인해 보세요."
- 버튼 1 (Primary): "이어서 견적 받기" → 모달 닫기
- 버튼 2 (텍스트 링크): "나중에 할게요" → 모달 닫기 (아무 동작 없음)

■ 스타일:
- 오버레이: 검정 50% 투명
- 모달: 흰 배경, border-radius 16px, padding 24px, 최대 320px
- 와이어프레임의 .modal-overlay, .modal-box, .modal-dismiss 스타일을 Tailwind로 변환
- Framer Motion으로 페이드인/아웃

■ 필수:
- 'use client' 필수
- any 타입 금지
- src/app/page.tsx에 이 모달을 추가하는 것도 함께 해줘
```

🔵 Phase 13: 정보/프로모션 페이지 (/info)

이 페이지의 역할: 장기렌터카의 장점, 프로모션, 인기 차종을 보여주고
간단 상담 폼으로 전환시키는 정보 페이지입니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 7장 "Frontend: Page 2 — 정보/프로모션"과
@docs/wireframe.html 의 Info 페이지 화면을 참고해서
src/app/info/page.tsx를 만들어줘.

■ 화면 구성 (위에서 아래):

[① Hero 배너]
- 그라디언트 배경: primary(#1B3A5C) → accent(#2E86C1)
- 높이: 60~70vh
- 제목: "장기렌터카, 카담에서 가장 쉽게"
- 부제: "나에게 맞는 조건으로 월 납부금을 비교해 보세요"
- CTA 버튼: "무료 견적 받기" (흰 배경, accent 텍스트) → router.push('/')

[② 장기렌터카 안내]
- 섹션 제목: "장기렌터카, 왜 좋을까요?"
- 구매 vs 렌트 비교 테이블:
  | 항목 | 구매 | 장기렌트 |
  | 초기 비용 | 수천만원 | 보증금만 |
  | 세금/보험 | 직접 관리 | 포함 |
  | 정비 | 직접 부담 | 포함 |
  | 차량 교체 | 매각 필요 | 반납/교체 |

[③ 이달의 프로모션]
- 섹션 제목: "이달의 프로모션"
- /api/promotions GET 호출하여 데이터 렌더링
- 가로 스크롤 카드 레이아웃 (flex, overflow-x-auto)
- 각 카드: 이미지(또는 placeholder) + 제목 + 설명
- 데이터 없으면 "현재 진행 중인 프로모션이 없습니다" 표시

[④ 인기 차종 견적]
- 섹션 제목: "인기 차종 견적 미리보기"
- @src/constants/vehicles.ts 에서 인기 6종 하드코딩 선택
  (예: 아반떼, 투싼, K5, 스포티지, 쏘렌토, 아이오닉5)
- 2열 그리드 카드: 차종명 + "월 XX만원~" + "견적 보기" 링크 → /cars/[slug]

[⑤ 간단 상담 폼]
- 제목: "1분 만에 상담 신청하기"
- 이름 + 연락처 + 개인정보 동의 체크박스
- "상담 신청" 버튼
- 제출 시 /api/consultation POST (step_completed: 0, inflow_page: '/info')

[⑥ Footer]
- 운영시간: "평일 09:00 ~ 18:00 (주말/공휴일 휴무)" (placeholder)
- 연락처: tap-to-call 전화번호 (NEXT_PUBLIC_PHONE_NUMBER, placeholder)
- 카카오톡: 채널 링크 (NEXT_PUBLIC_KAKAO_CHANNEL_URL, placeholder)
- 개인정보 처리방침: /privacy 링크
- 사업자 정보: "(사업자 등록 후 기입)" placeholder
- "© 카담(CADAM)" 표기

■ 스타일:
- 와이어프레임의 .hero, .section, .compare-table, .promo-scroll,
```

```
  .pop-grid, .footer 스타일을 Tailwind로 변환
- 모바일 퍼스트, Tailwind CSS만 사용
- 인라인 style 금지

■ 필수:
- 'use client' (프로모션 API 호출, 폼 상태관리 필요)
- any 타입 금지
- Footer는 별도 컴포넌트로 분리해도 됨 (다른 페이지에서 재사용)
```

🔵 Phase 14: SEO 차종별 상세 페이지 (/cars/[slug])

이 페이지의 역할: 45개 차종 각각의 SEO 랜딩 페이지입니다.
구글/네이버 검색에서 "아반떼 장기렌트"로 유입되는 트래픽을 받습니다.
ISR(Incremental Static Regeneration)로 1시간마다 자동 재생성됩니다.

Cursor Composer 프롬프트:

```
@docs/PRD.md 6장 "SEO 차종별 상세 페이지"를 참고해서
다음 파일들을 만들어줘:

■ 1) src/app/cars/[slug]/page.tsx (메인 페이지)

[ISR 설정]
- export const revalidate = 3600 (1시간)
- generateStaticParams(): VEHICLE_LIST.map(v => ({ slug: v.slug }))
  → 45개 페이지 사전 생성

[★ Next.js 15 필수 ★]
- params는 Promise 타입! 반드시 await 처리:
  export async function generateMetadata({ params }: { params: Promise<{ slug: string }
> }) {
    const { slug } = await params;
    ...
  }
  export default async function CarPage({ params }: { params: Promise<{ slug: string }
> }) {
    const { slug } = await params;
    ...
  }

[메타데이터]
- title: vehicle.seoTitle
- description: vehicle.seoDescription
- openGraph: { title, images: ['/cars/{imageKey}.webp'] }
- canonical URL 설정
- JSON-LD Product 스키마 (구글 리치 결과용)

[페이지 구성]
① Hero: 차량 이미지 + 차종명 + 세그먼트/연료 태그 + "월 XX만원부터~"
② 가격 비교표: Supabase price_ranges 조회, 계약기간(행) × 주행거리(열)
③ 장기렌트 장점 (하드코딩 공통 콘텐츠)
④ CTA: "무료 견적 받기" → '/' + "카카오톡 상담" + "전화 상담"
⑤ 관련 차종: 같은 카테고리 최대 4개 (VEHICLE_LIST에서 filter, DB 쿼리 없음)

[데이터 없는 경우]
- price_ranges 매칭 실패: "상담 문의" 표시
- 차량 이미지 없음: placeholder 이미지 또는 회색 박스

■ 2) src/components/cars/CarHero.tsx
- 차량 이미지(Next.js Image 컴포넌트) + 차종명 + 태그 + 최저가
- 와이어프레임의 .seo-hero 스타일을 Tailwind로 변환

■ 3) src/components/cars/PriceCompareTable.tsx
- props: price_ranges 배열
- 계약기간(행) × 주행거리(열) 테이블
- 와이어프레임의 .price-table 스타일을 Tailwind로 변환

■ 4) src/components/cars/RelatedCars.tsx
- props: currentVehicle
- VEHICLE_LIST에서 같은 category, 자기 자신 제외, 최대 4개
```

```
- 각 카드 → /cars/[slug] 링크

■ 규칙:
- page.tsx는 서버 컴포넌트 ('use client' 없음) — Supabase 직접 조회
- 하위 컴포넌트 중 인터랙티브한 것만 'use client'
- any 타입 금지
- 차량 데이터는 @src/constants/vehicles.ts 사용
- 가격 데이터는 Supabase 조회 (프론트엔드 계산 금지)
```

🔵 Phase 15: 개인정보 처리방침 (/privacy)

Cursor Composer 프롬프트:

```
@docs/PRD.md 12장 "개인정보 처리방침" 내용을 기반으로
src/app/privacy/page.tsx를 만들어줘.

■ 페이지 구성 (정적 콘텐츠, API 호출 없음):

1. 제목: "개인정보 처리방침"
2. 수집하는 개인정보 항목
   - 필수: 이름, 연락처(휴대폰 번호)
   - 자동 수집: 접속 기기 정보, 유입 경로, IP 해시(비식별 처리)
3. 수집 및 이용 목적
   - 장기렌터카 상담 견적 제공 및 상담사 연결
   - 서비스 개선을 위한 통계 분석 (비식별 처리)
4. 보유 및 이용 기간
   - 상담 완료 후 1년간 보유 후 파기
5. 동의 거부 권리
6. 개인정보 처리 위탁 (Resend, Vercel, Supabase)
7. 개인정보 보호책임자: "(사업자 정보 확정 후 기입)"

■ 스타일:
- 깔끔한 문서 레이아웃, max-width 제한
- 제목은 primary 색상, 소제목은 font-weight 600
- 서버 컴포넌트 (정적이므로 'use client' 불필요)
- Tailwind CSS만 사용
```

🔵 Phase 16: Middleware (쿠키 수집)

Cursor Composer 프롬프트:

```
@docs/PRD.md 8.1장 "Middleware" 코드를 참고해서
src/middleware.ts를 만들어줘.

■ PRD에 코드가 이미 있으므로 그대로 구현:

1. User-Agent → device_type 쿠키
   - request.headers.get('user-agent')에서 읽기
   - /Mobile|Android|iPhone/i 매칭 → 'Mobile' : 'Desktop'
   - 쿠키 maxAge: 30일

2. Referer → referrer 쿠키
   - request.headers.get('referer')에서 읽기
   - 이미 referrer 쿠키가 있으면 덮어쓰지 않음
   - 쿠키 maxAge: 30일

3. UTM → utm_source 쿠키
   - request.nextUrl.searchParams.get('utm_source')
   - 쿠키 maxAge: 30일

4. inflow_page 쿠키
   - request.nextUrl.pathname
   - 쿠키 maxAge: 24시간

■ matcher 설정:
export const config = {
  matcher: ['/', '/info', '/cars/:path*'],
};

■ 규칙:
- NextRequest, NextResponse 사용
- document.referrer, navigator.userAgent 사용 금지
  (서버사이드 Middleware에서 브라우저 객체 접근 불가)
- any 타입 금지
```

🔵 Phase 17: Admin 로그인 + 대시보드

17-1. Admin 레이아웃 + 인증 체크

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.1장 "인증"을 참고해서
src/app/admin/layout.tsx를 만들어줘.

■ 역할:
- /admin 하위 모든 페이지의 공통 레이아웃
- Supabase Auth 세션 체크
- 미인증 시 → /admin/login으로 리다이렉트
- 인증 시 → children 렌더링 + 상단 네비게이션 바

■ 네비게이션 바:
- 좌측: "카담 Admin" 로고
- 메뉴: 대시보드 | 상담 관리 | 프로모션 | 가격표
- 우측: 로그아웃 버튼

■ Supabase Auth:
- @src/lib/supabase.ts 사용
- 세션 만료: 24시간
- supabase.auth.getSession()으로 체크

■ 필수:
- 'use client' (인증 상태 체크 + 네비게이션)
- any 타입 금지
- Admin 전체는 dynamic import로 분리 (번들 최적화)
```

17-2. Admin 로그인 페이지

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.1장을 참고해서
src/app/admin/login/page.tsx를 만들어줘.

■ 화면:
- 중앙 정렬 로그인 폼
- "카담 관리자 로그인" 제목
- 이메일 입력 (text input)
- 비밀번호 입력 (password input)
- "로그인" 버튼 (accent 배경, 전체 폭)
- 로딩 상태: 버튼 내 스피너
- 에러: "이메일 또는 비밀번호가 올바르지 않습니다" 빨간 텍스트

■ 로직:
- supabase.auth.signInWithPassword({ email, password })
- 성공 시 → router.push('/admin')
- 실패 시 → 에러 메시지 표시

■ 필수:
- 'use client' 필수
- any 타입 금지
- 이 페이지는 admin/layout.tsx의 인증 체크에서 제외되어야 함
```

17-3. Admin 대시보드

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.2장 "대시보드"를 참고해서
src/app/admin/page.tsx를 만들어줘.

■ 요약 카드 (상단, 가로 배치 또는 2열 그리드):

| 카드명 | Supabase 쿼리 | 색상 |
|--------|-------------|------|
| 오늘 신청 | COUNT(created_at >= 오늘 00:00) | accent |
| 미처리 | COUNT(status = 'pending') | danger |
| HOT 리드 | COUNT(lead_score >= 80 AND status = 'pending') | danger (강조) |
| 이번 주 신청 | COUNT(created_at >= 이번 주 월요일) | success |
| 전환율 | (contracted / total) * 100 (%) | warning |

■ 오늘의 콜백 리스트 (카드 아래):
- callback_time이 오늘 날짜인 건들 자동 표시
- 각 행: 고객명 + 차종 + 콜백 시간 + 메모 미리보기
- 전화번호 클릭 → tel: 링크

■ 하단 빠른 메뉴:
- 상담 관리 → /admin/consultations
- 프로모션 관리 → /admin/promotions
- 가격표 관리 → /admin/prices

■ 스타일:
- 와이어프레임의 .admin-cards, .admin-stat 스타일을 Tailwind로 변환
- HOT 리드 카드: border-l-4 border-danger
- Tailwind CSS만 사용

■ 필수:
- 'use client' (실시간 데이터 로드)
- any 타입 금지
- @src/components/admin/CallbackList.tsx 분리 가능
```

🔵 Phase 18: Admin 상담 관리 CRM

이것은 Admin에서 가장 복잡한 페이지입니다. 2~3단계로 나누어 작업하세요.

18-1. 상담 테이블 + 리드 뱃지

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.3장 "상담 관리"를 참고해서 다음 파일들을 만들어줘:

■ 1) src/components/admin/LeadBadge.tsx
- props: score (number)
- 80~100: 🔴 HOT (bg-red-50, text-danger)
- 50~79: 🟠 WARM (bg-orange-50, text-warning)
- 20~49: 🟡 COOL (bg-yellow-50, text-yellow-700)
- 0~19: ⚪ COLD (bg-gray-50, text-gray-500)

■ 2) src/components/admin/ConsultationTable.tsx
- Supabase에서 consultations 전체 조회 (authenticated)
- 테이블 컬럼 11개 (PRD 9.3장 테이블 그대로):
  신청일 | 리드(LeadBadge) | 이름 | 연락처(tap-to-call) | 차종 |
  계약조건 | 예상금액 | 상태(드롭다운) | 결과(드롭다운) | 상담사(드롭다운) | 콜백

- HOT 리드 하이라이팅:
  lead_score >= 80: bg-red-50 border-l-4 border-red-500
  lead_score >= 50: bg-orange-50

- 상태 드롭다운: pending/consulting/completed
  → 변경 시 즉시 Supabase UPDATE
- 결과 드롭다운: null/contracted/competitor/hold/no_answer/cancelled
  → 변경 시 즉시 Supabase UPDATE
- 상담사 드롭다운: 직접 입력 또는 선택

- 행 클릭 시 → ConsultationDetail 상세 패널 오픈

■ 필터 (테이블 상단):
- 상태별: 전체 / 미처리 / 상담중 / 완료
- 리드 등급별: 전체 / HOT / WARM / COOL / COLD
- 날짜 범위: 시작일 ~ 종료일
- 차종별 / 상담 결과별 / 상담사별
- "HOT 리드" 바로가기 (lead_score >= 80 AND status = 'pending')
- "콜백 예정" 바로가기 (callback_time IS NOT NULL)

■ 정렬:
- 기본: 리드 점수 높은 순 (lead_score DESC)
- 선택 가능: 최신순 (created_at DESC) / 콜백 시간순

■ 스타일:
- 와이어프레임의 .admin-row, .status-badge, .lead-badge 스타일을 Tailwind로 변환
- 반응형: 모바일에서는 주요 컬럼만 표시, 가로 스크롤

■ 필수:
- 'use client' 필수
- any 타입 금지
```

18-2. 상담 상세 패널 (CRM)

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.3장 "상세 패널"을 참고해서
src/components/admin/ConsultationDetail.tsx를 만들어줘.

■ 트리거: ConsultationTable에서 행 클릭 시 슬라이드 오픈 (우측 또는 하단)

■ 표시 내용:
[고객 선택 내역]
- 선택 경로 (차종/예산)
- 차종: 브랜드 + 모델 + 트림
- 계약 기간, 연간 주행거리
- 보증금, 선납금 비율
- 월 예산 (budget 경로인 경우)

[유입 정보]
- utm_source, referrer, inflow_page, device_type

[리드 점수]
- 숫자 + LeadBadge 컴포넌트

[CRM — 상담 메모]
- textarea (여러 줄)
- "저장" 버튼 → Supabase consultations UPDATE (memo 필드)
- 저장 성공 시 Toast "메모가 저장되었습니다"

[CRM — 재연락 시간]
- <input type="datetime-local" />
- "저장" 버튼 → Supabase UPDATE (callback_time 필드)
- 저장 성공 시 Toast "콜백 시간이 설정되었습니다"

[CRM — 상담 결과]
- 드롭다운: 미정/계약완료/타사계약/보류/부재/취소
- PRD의 consult_result 허용 값과 Admin 표시:
  contracted → 🟢 계약완료
  competitor → 🔴 타사계약
  hold → 🟡 보류
  no_answer → ⚫ 부재
  cancelled → ⚪ 취소
- "저장" 버튼 → Supabase UPDATE

[알림 발송 이력]
- notification_log 테이블에서 해당 consultation_id 조회
- 각 행: 채널(email) + 상태(sent/failed) + 시각

[상태 변경]
- pending → consulting → completed 단계별 토글

■ 스타일:
- 와이어프레임의 .crm-panel 스타일을 Tailwind로 변환
- Framer Motion 슬라이드 애니메이션
- "닫기" 버튼 상단 우측
```

```
■ 필수:
- 'use client' 필수
- any 타입 금지
- DOMPurify로 memo 입력 XSS 방지 (Admin 입력이지만 보안 습관)
```

18-3. 상담 관리 페이지 조립

Cursor Composer 프롬프트:

```
src/app/admin/consultations/page.tsx를 만들어서
@src/components/admin/ConsultationTable.tsx 와
@src/components/admin/ConsultationDetail.tsx 를 조합해줘.

- ConsultationTable에서 행 선택 시 ConsultationDetail이 우측 슬라이드로 열림
- 선택된 consultation의 id를 Detail에 전달
- 'use client' 필수
- any 타입 금지
```

🔵 Phase 19: Admin 프로모션 + 가격표 관리

19-1. 프로모션 관리

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.4장 "프로모션 관리"를 참고해서 다음 파일들을 만들어줘:

■ 1) src/app/admin/promotions/page.tsx
- 목록 뷰: 카드형 레이아웃
- 각 카드: 이미지(또는 placeholder) + 제목 + 활성 토글 + 노출 기간
- "새 프로모션 추가" 버튼 → PromotionForm 모달 오픈
- 카드 클릭 → PromotionForm 수정 모드로 오픈

■ 2) src/components/admin/PromotionForm.tsx
- 등록/수정 폼 (모달 또는 슬라이드 패널):
  제목 (text input, 필수)
  설명 (textarea)
  이미지 URL (text input)
  링크 URL (text input)
  시작일 (date picker, 필수)
  종료일 (date picker, 필수)
  정렬 순서 (number input)
  활성 여부 (toggle switch, 필수)

- 등록: Supabase promotions INSERT
- 수정: Supabase promotions UPDATE
- 삭제: 확인 모달 "정말 삭제하시겠습니까?" → Supabase DELETE

■ 규칙:
- 'use client' 필수
- any 타입 금지
- Tailwind CSS만 사용
```

19-2. 가격표 관리

Cursor Composer 프롬프트:

```
@docs/PRD.md 9.5장 "가격표 관리"를 참고해서 다음 파일들을 만들어줘:

■ 1) src/app/admin/prices/page.tsx
- 목록 뷰: 브랜드 > 차종 아코디언 트리 구조
  └ 현대 (접기/펼치기)
    └ 캐스퍼 - 36개월/1만km: 25~35만원
    └ 캐스퍼 - 48개월/2만km: 30~40만원
    ...
  └ 기아
    ...
- "새 가격 추가" 버튼 → PriceEditor 모달 오픈
- 행 클릭 → PriceEditor 수정 모드

■ 2) src/components/admin/PriceEditor.tsx
- 등록/수정 폼:
  브랜드: select (현대/기아/제네시스, @src/constants/vehicles.ts에서 Brand 타입)
  차종: select (선택 브랜드의 차종 목록, VEHICLE_LIST에서 filter)
  계약기간: select (36/48/60)
  주행거리: select (10000/20000/30000/40000)
  최소 월 납부금: number input (원)
  최대 월 납부금: number input (원)
  조건별 상세: JSON editor (textarea, jsonb 형태)

- 등록: Supabase price_ranges INSERT
- 수정: Supabase price_ranges UPDATE
  → 수정 성공 후 자동으로 /api/revalidate POST 호출
    (해당 차종의 slug 전달 → SEO 페이지 즉시 재생성)

- CSV 업로드 기능:
  파일 선택 → 파싱 (형식: brand, model, months, km, min, max)
  → 미리보기 테이블 → "업로드" 클릭 시 다건 INSERT/UPDATE

■ 규칙:
- 브랜드 선택 시 → 해당 브랜드 차종만 동적 필터링
- conditions JSON은 파싱만 허용, 프론트엔드에서 수학적 계산 절대 금지
- 'use client' 필수
- any 타입 금지
```

🔵 Phase 20: GA4 이벤트 연동

Cursor Composer 프롬프트:

```
@docs/PRD.md 10장 "GA4 Analytics & Event Tracking"을 참고해서
기존 컴포넌트들에 GA4 이벤트를 추가해줘.

■ 전제: @src/lib/gtag.ts 가 이미 있어야 함

■ 이벤트 목록 (해당 컴포넌트에 각각 추가):

1. page_view → src/app/layout.tsx (모든 페이지)
2. step_view → StepLayout.tsx (Step 화면 진입 시)
   파라미터: step_number, step_name
3. step_complete → 각 Step 컴포넌트 (선택 완료 시)
   파라미터: step_number, selected_value
4. step_back → StepLayout.tsx (이전 Step 이동 시)
   파라미터: from_step, to_step
5. leave_intent → LeaveModal.tsx (모달 표시 시)
   파라미터: current_step
6. leave_intent_stay → LeaveModal.tsx ("이어서 견적 받기" 클릭)
7. leave_intent_exit → LeaveModal.tsx ("나중에 할게요" 클릭)
8. form_submit → Step6Contact.tsx (폼 제출 시도)
   파라미터: car_brand, car_model
9. form_submit_success → Step6Contact.tsx (신청 완료)
   파라미터: car_brand, car_model, lead_score
10. form_submit_error → Step6Contact.tsx (신청 실패)
    파라미터: error_type
11. result_kakao_click → result/page.tsx (카카오톡 클릭)
12. result_call_click → result/page.tsx (전화 클릭)
13. result_retry → result/page.tsx ("다른 차량도" 클릭)
14. seo_page_view → cars/[slug]/page.tsx
    파라미터: car_slug, car_brand
15. seo_cta_click → cars/[slug]/page.tsx (CTA 클릭)
    파라미터: car_slug, cta_type
16. info_cta_click → info/page.tsx (CTA 클릭)
    파라미터: section
17. quick_form_submit → info/page.tsx (간단 폼 제출)

■ 호출 방식:
gtag.ts의 함수를 import해서 해당 이벤트 발생 시점에 호출
예: gtag.event('step_complete', { step_number: 3, selected_value: '48개월' })

■ 규칙:
- NEXT_PUBLIC_GA_ID가 없으면 이벤트 발송하지 않음 (개발 환경 대비)
- any 타입 금지
- 기존 컴포넌트 로직을 변경하지 말고, 이벤트 호출만 추가
```

🔵 Phase 21: 테스트 + Vercel 배포

21-1. 빌드 에러 수정

Cursor 터미널에서 실행:

```
# TypeScript 에러 확인
npx tsc --noEmit

# ESLint 확인
npx next lint

# 프로덕션 빌드 테스트
npm run build
```

에러가 나오면 Cursor Chat(`Cmd+L`)에 복사 붙여넣기:

```
npm run build에서 이런 에러들이 나왔어:
[에러 메시지 전체 붙여넣기]
모두 수정해줘.
```

21-2. 기능 테스트 체크리스트

Cursor Composer 프롬프트 (테스트 계획 생성):

```
카담 프로젝트의 기능 테스트 체크리스트를 만들어줘.

다음 항목을 포함:
1. Step 1~6 플로우 정상 진행 (차종 경로 / 예산 경로 모두)
2. Step 2A에서 브랜드/카테고리 필터 작동
3. Step 6 폼 유효성 검사 (빈 이름, 잘못된 전화번호, 미동의)
4. API /api/consultation POST 성공/실패
5. Rate Limit 테스트 (3회 초과 시 429)
6. 결과 페이지 가격 표시 / 미표시
7. /info 페이지 렌더링
8. /cars/[slug] SEO 페이지 (아무 차종 하나)
9. /privacy 페이지 렌더링
10. Admin 로그인 → 대시보드 → 상담 목록 → 상세 패널
11. Admin 프로모션 CRUD
12. Admin 가격표 CRUD → Revalidation
13. 이탈 방지 모달 (Step 2 이상에서)
14. Toast 에러 메시지
15. 모바일 반응형 (360px 폭)
```

21-3. 보안 최종 점검

Cursor Composer 프롬프트:

```
카담 프로젝트 전체에서 보안 점검을 해줘:

1. SUPABASE_SERVICE_ROLE_KEY가 클라이언트 컴포넌트에서 사용되는 곳이 없는지
   (NEXT_PUBLIC_ 접두사가 아닌 환경변수가 프론트엔드에 노출되면 안 됨)
2. leadScore 계산이 서버사이드(API Route)에서만 호출되는지
3. localStorage 직접 접근이 없는지 (Zustand persist만 허용)
4. console.log가 프로덕션 코드에 남아있는지
5. any 타입이 사용된 곳이 있는지
6. 인라인 style={{}} 이 사용된 곳이 있는지
7. kakao_sent 관련 로직이 작성되어 있는지 (Phase 2 전용, 있으면 삭제)
8. .env.local이 .gitignore에 포함되어 있는지
```

21-4. Vercel 배포

① GitHub 저장소 생성 + 코드 Push:

```
# 프로젝트 루트에서
git init
git add .
git commit -m "카담 MVP 초기 배포"

# GitHub에서 새 저장소 생성 후
git remote add origin https://github.com/[계정명]/cadam.git
git branch -M main
git push -u origin main
```

② Vercel 연결:

1. https://vercel.com 접속 → Sign Up (GitHub 계정으로)
2. "Add New Project" → GitHub 저장소 "cadam" 선택
3. Framework Preset: "Next.js" (자동 감지)
4. **환경변수 설정 (가장 중요!):**

`.env.local` 의 모든 환경변수를 Vercel에 하나씩 입력:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- UPSTASH_REDIS_REST_URL
- UPSTASH_REDIS_REST_TOKEN
- RESEND_API_KEY
- ADMIN_EMAIL
- NEXT_PUBLIC_GA_ID
- REVALIDATION_SECRET

- NEXT_PUBLIC_KAKAO_CHANNEL_URL
- NEXT_PUBLIC_PHONE_NUMBER
5. "Deploy" 클릭 → 빌드 + 배포 (2~3분)
6. 배포 완료 후 생성된 URL(예: cadam.vercel.app)에서 확인

③ 도메인 연결 (선택):
1. Vercel → Settings → Domains
2. cadam.co.kr 입력 (도메인 구매 후)
3. DNS 설정 지시에 따라 네임서버 또는 CNAME 설정

④ 배포 후 확인:

```
배포된 URL에서 다음을 확인:
- Step 1~6 플로우 정상 작동
- 상담 신청 시 Supabase에 데이터 저장
- 관리자 이메일(iori21y@gmail.com)로 알림 수신
- Admin 페이지 로그인 + 대시보드
- /cars/avante 같은 SEO 페이지 접근
- 모바일에서 정상 표시
```

📋 Phase 10~21 진행 체크리스트

Phase 10: API Routes

- [ ] /api/consultation (10단계 처리 흐름)
- [ ] /api/price-range
- [ ] /api/promotions
- [ ] /api/revalidate
- [ ] curl로 API 테스트 성공

Phase 11: 결과 페이지

- [ ] budget 경로 화면
- [ ] car 경로 + 가격 표시
- [ ] CTA 3개 (카카오/전화/다른차량)

Phase 12: Toast + 이탈 모달

- [ ] Toast 3종 (error/success/warning)
- [ ] 이탈 모달 (Step 2 이상, 세션당 1회)

Phase 13: 정보 페이지

- [ ] Hero + 비교표 + 프로모션 + 인기차종 + 간단폼 + Footer

Phase 14: SEO 페이지

- [ ] /cars/[slug] ISR 작동
- [ ] 메타데이터 + JSON-LD
- [ ] 가격 비교표 + 관련 차종

Phase 15: 개인정보 처리방침

- [ ] /privacy 정적 페이지

Phase 16: Middleware

- [ ] 쿠키 4종 (device_type, referrer, utm_source, inflow_page)

Phase 17: Admin 인증 + 대시보드

- [ ] 로그인 작동
- [ ] 미인증 리다이렉트
- [ ] 요약 카드 5개 + 콜백 리스트

Phase 18: Admin CRM

- [ ] 상담 테이블 11컬럼
- [ ] HOT 리드 하이라이팅
- [ ] 상세 패널 (메모/콜백/결과)
- [ ] 필터 + 정렬

Phase 19: Admin 프로모션 + 가격표

- [ ] 프로모션 CRUD
- [ ] 가격표 CRUD + Revalidation
- [ ] CSV 업로드

Phase 20: GA4

- [ ] 17개 이벤트 연동

Phase 21: 배포

- [ ] npm run build 에러 없음
- [ ] 기능 테스트 통과
- [ ] 보안 점검 통과
- [ ] Vercel 배포 + 환경변수 설정
- [ ] 배포 URL에서 최종 확인

⚠️ Cursor AI 사용 시 핵심 팁

팁 1: 작은 단위로 지시하라

```
❌ "PRD 전체를 보고 프로젝트를 완성해줘"
✅ "PRD 5.2장의 Step 3을 만들어줘. @docs/wireframe.html 참고."
```

팁 2: @ 참조를 항상 사용하라

```
❌ "차량 데이터를 사용해서..."
✅ "@src/constants/vehicles.ts 의 VEHICLE_LIST를 사용해서..."
```

팁 3: 규칙을 명시적으로 상기시켜라

```
"Tailwind CSS만 사용, 인라인 style 금지.
any 타입 금지, interface 사용.
'use client' 필수."
```

팁 4: 에러가 나면 에러 메시지를 그대로 복사해서 물어봐라

```
"터미널에서 이런 에러가 나왔어: [에러 메시지 붙여넣기]
이거 어떻게 해결해?"
```

팁 5: AI가 만든 코드를 바로 수락하지 말고 검토하라

특히 확인할 것:
- `any` 타입이 쓰이지 않았는지
- `style={{}}` 인라인 스타일이 없는지

- `import` 경로가 올바른지 (`@/` 접두사)
- 차량 데이터를 하드코딩하지 않고 `VEHICLE_LIST`를 참조하는지

팁 6: 자주 저장하고, 자주 확인하라

```
# 변경사항 저장 후 개발 서버 실행
npm run dev
# http://localhost:3000 에서 화면 확인
```

팁 7: Git으로 중간 저장하라

```
# 큰 기능이 완성될 때마다
git add .
git commit -m "Step 1-3 컴포넌트 완성"
```

문제가 생기면 `git checkout .`으로 마지막 저장 시점으로 되돌릴 수 있습니다.

📋 체크리스트 (진행 확인용)

Phase 0: 사전 준비

- [ ] Node.js LTS 설치 완료
- [ ] Git 설치 완료
- [ ] Cursor AI 설치 + 계정 생성

Phase 1: 프로젝트 생성

- [ ] `npx create-next-app@latest cadam` 실행
- [ ] `npm run dev`로 기본 화면 확인

Phase 2: Cursor 환경설정

- [ ] `.cursorrules` 파일 배치
- [ ] `.cursor/rules` 파일 배치
- [ ] `docs/PRD.md` 배치
- [ ] `docs/wireframe.html` 배치

Phase 3: 패키지 설치

- [ ] zustand, framer-motion 설치
- [ ] supabase 관련 패키지 설치
- [ ] upstash, resend, zod, dompurify 설치

Phase 4: Tailwind 설정

- [ ] 커스텀 색상 6개 등록
- [ ] Pretendard 폰트 설정

Phase 5: 외부 서비스

- [ ] Supabase 프로젝트 생성 + 키 발급
- [ ] Upstash Redis 생성 + 키 발급
- [ ] Resend 가입 + API 키 발급
- [ ] GA4 속성 생성 + 측정 ID 발급
- [ ] `.env.local` 파일 작성 완료

Phase 6: 폴더 구조

- [ ] PRD 13장 기준 폴더 전부 생성

Phase 7: 핵심 코드 생성

- [ ] vehicles.ts (45종 확인)
- [ ] quoteStore.ts
- [ ] lib/ 유틸리티 7개 파일
- [ ] hooks/ 2개 파일

Phase 8: 데이터베이스

- [ ] Supabase 테이블 4개 생성
- [ ] RLS 정책 적용
- [ ] 인덱스 7개 생성
- [ ] 관리자 계정 생성

Phase 9: 첫 화면

- [ ] layout.tsx 수정 (폰트, GA4)
- [ ] StepLayout + ProgressBar

- [ ] Step 1 ~ Step 6 컴포넌트
- [ ] page.tsx에서 Step 조립
- [ ] `npm run dev`로 전체 플로우 확인!